

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海站

质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome

从像素到语义：大模型 + CV构建全终端UI检测下一代技术

讲 师：徐崴 (腾讯音乐娱乐集团)

质效革新·智领未来

目录

CONTENT

1

高成本、低效率：传统UI自动化的困局

高成本、低效率, 多年来UI自动化一直被诟病, 如何破局?

2

构建纯视觉UI检测平台Page eyes 1.0

通过AI+纯视觉方案, 从UI面内容校验入手, 搭建智能UI检测系统

3

从RPA到智能体探索：Page eyes Agent 2.0

打造最懂你的GUI Agent--实现0代码 “智” 动化方案

4

总结和展望

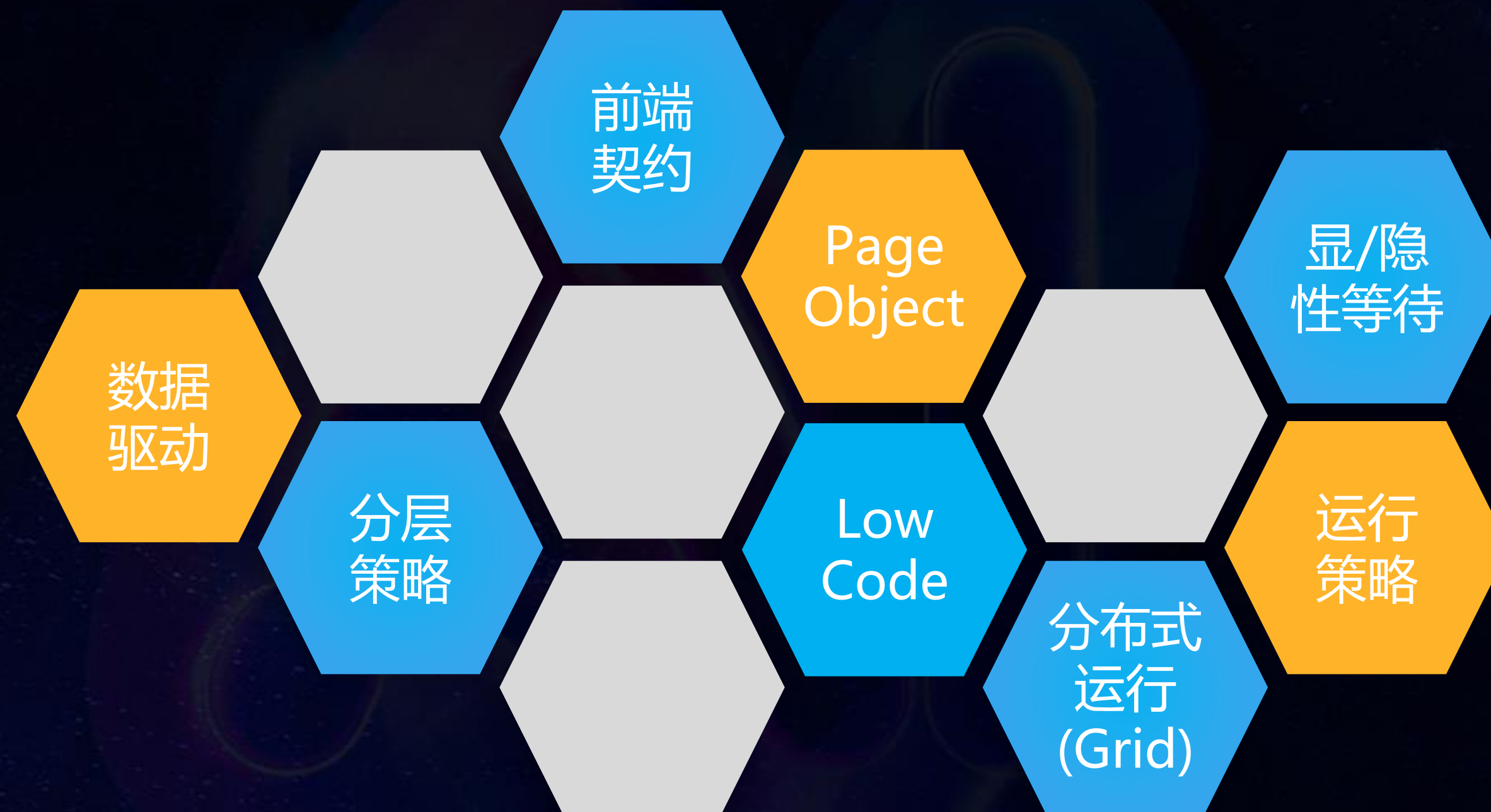
未来可期, 还是未来已来?

1

高成本、低效率：传统UI自动化的困局

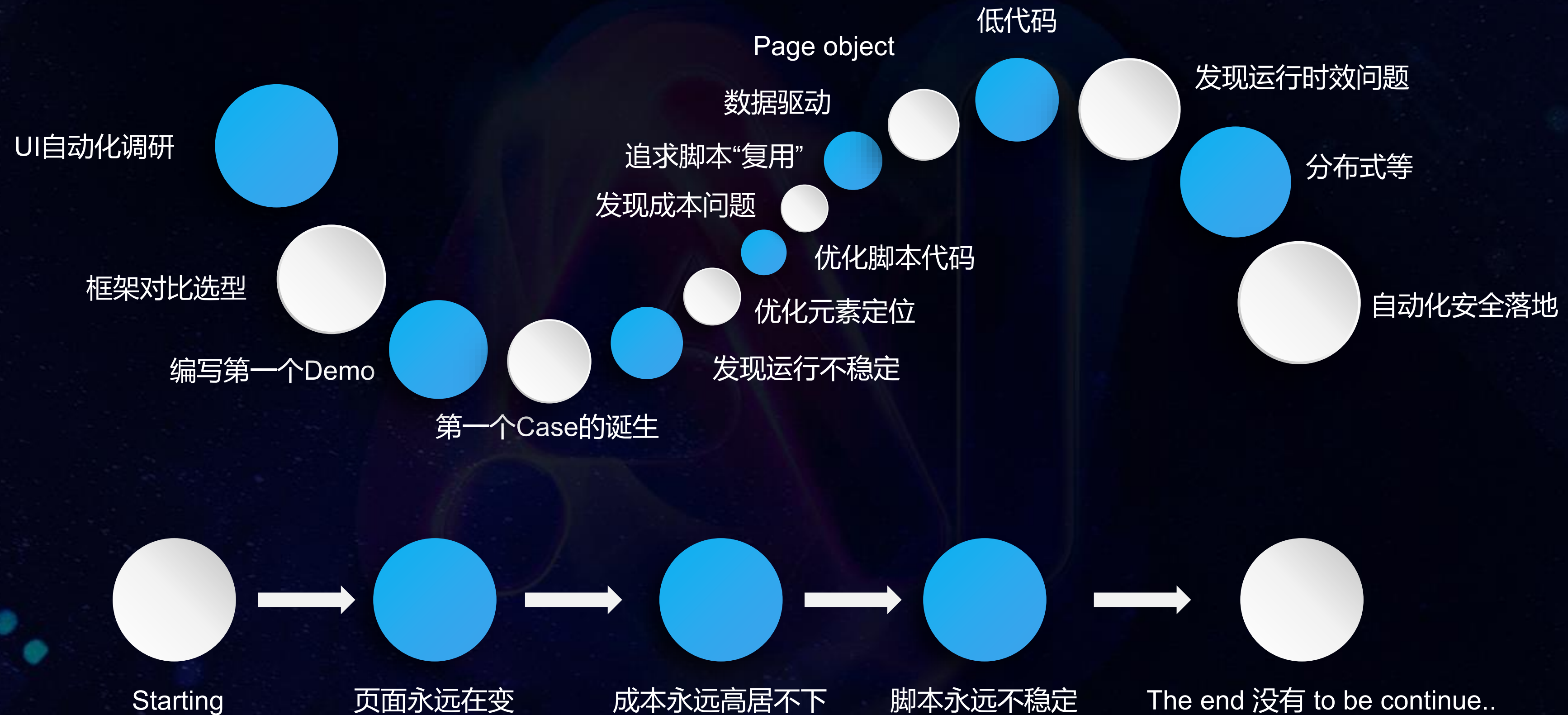
高成本、低效率, 多年来UI自动化一直被诟病, 如何破局?

那些年, 我们一起 “追” 过的UI自动化



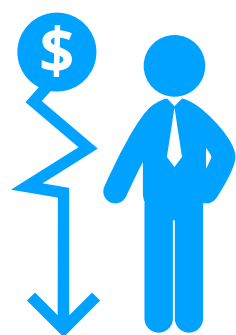
UI自动化从诞生的第一天, 我们一直在探索 ...

理想 VS 现实



UI自动化面临的困局

高成本



- UI频繁变更导致用例失效
- 元素定位和识别的成本
- 需求快速变化的影响
- 高级脚本和框架掌握

不稳定



- 环境依赖性强
- 动态内容难处理
- 测试环境一致性要求
- 误报率高

低效性

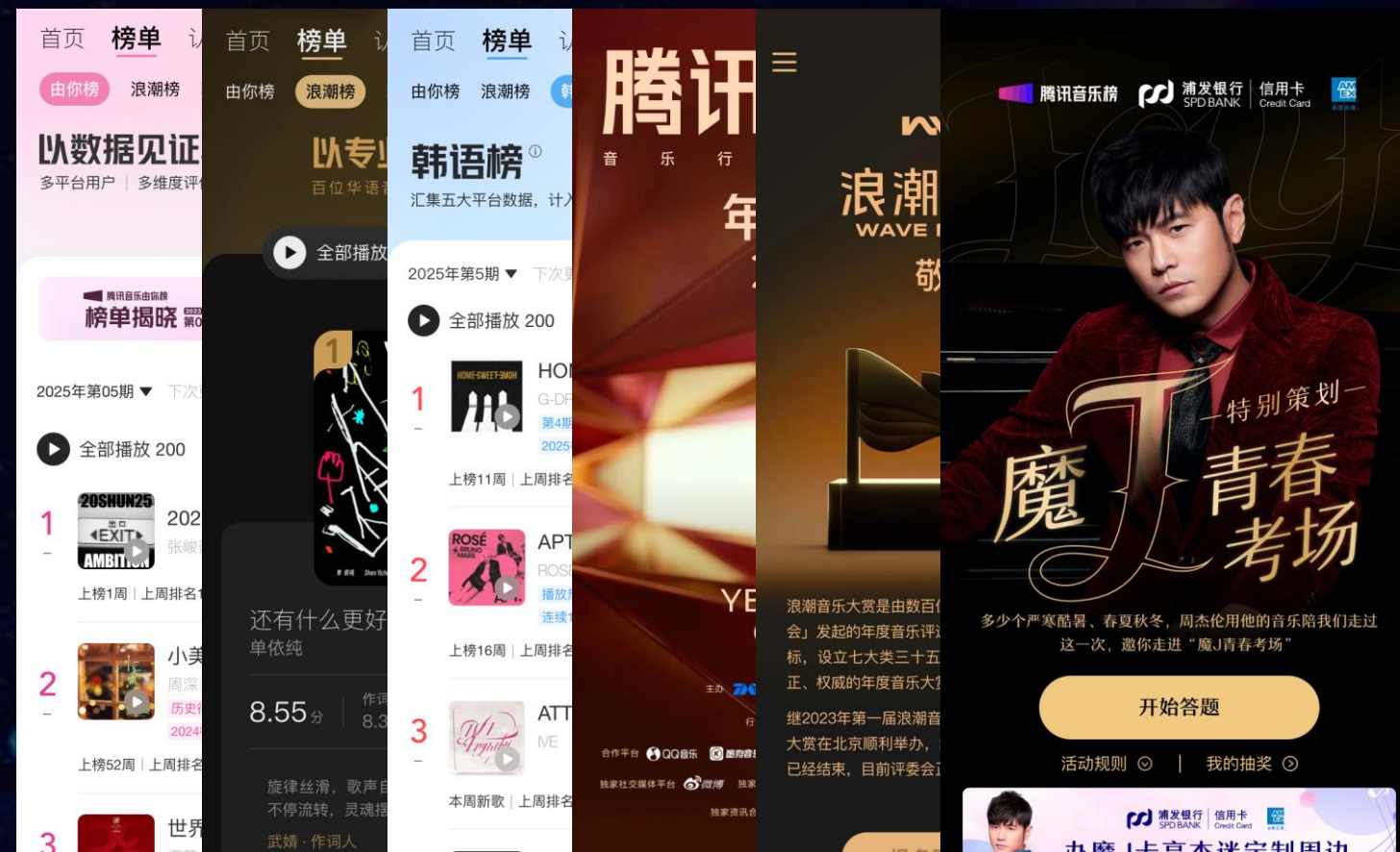


- 初期脚本开发耗时长
- 步骤多耗时长
- 敏捷迭代适配难度大
- 排错耗时长

质量面临的挑战

腾讯音乐榜 | 商业化运营活动

多端、多平台、多用户类型、多依赖、多页面



背景

* 问题分布:



* 问题特点:

样式、布局错乱

接口报错、JS 报错

无数据展示、加载白屏

页面崩溃

体验差

影响使用

无法使用

挑战

2

构建纯视觉UI检测平台Page eyes 1.0

通过AI+纯视觉方案, 从UI面内容校验入手, 搭建智能UI检测系统

从元素属性到纯视觉



纯视觉UI检测(Page eyes)初步构想

模拟人眼

通过视觉方案去快速检查页面，更接近用户访问视角，同时异常信息也可直观的展示给测试人员，不需要繁琐的日志排查

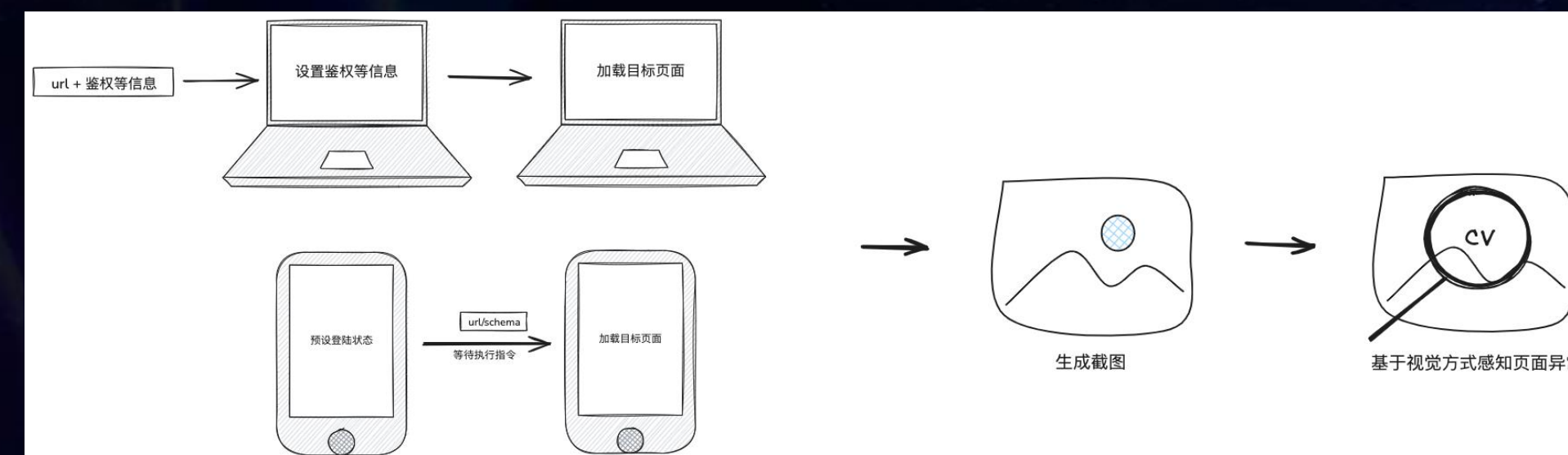


另辟蹊径，以低成本的自动化补位方式发现和拦截页面异常的致命问题，做到质量与效率的平衡

维护轻量

页面截图

页面检测



已有的自动化能力会消耗测试工程师较多的时间用于编写和调试用例，作为传统自动化测试补位的一个能力需尽可能减少额外的工作量

打开目标页面

元素视觉识别

微交互执行

页面检测

Page eyes检测策略

页面严重问题

页面白屏

页面报错

样式布局错乱

应对策略

线下执行拦截 | 线上定时巡检

检查页面是否有白屏

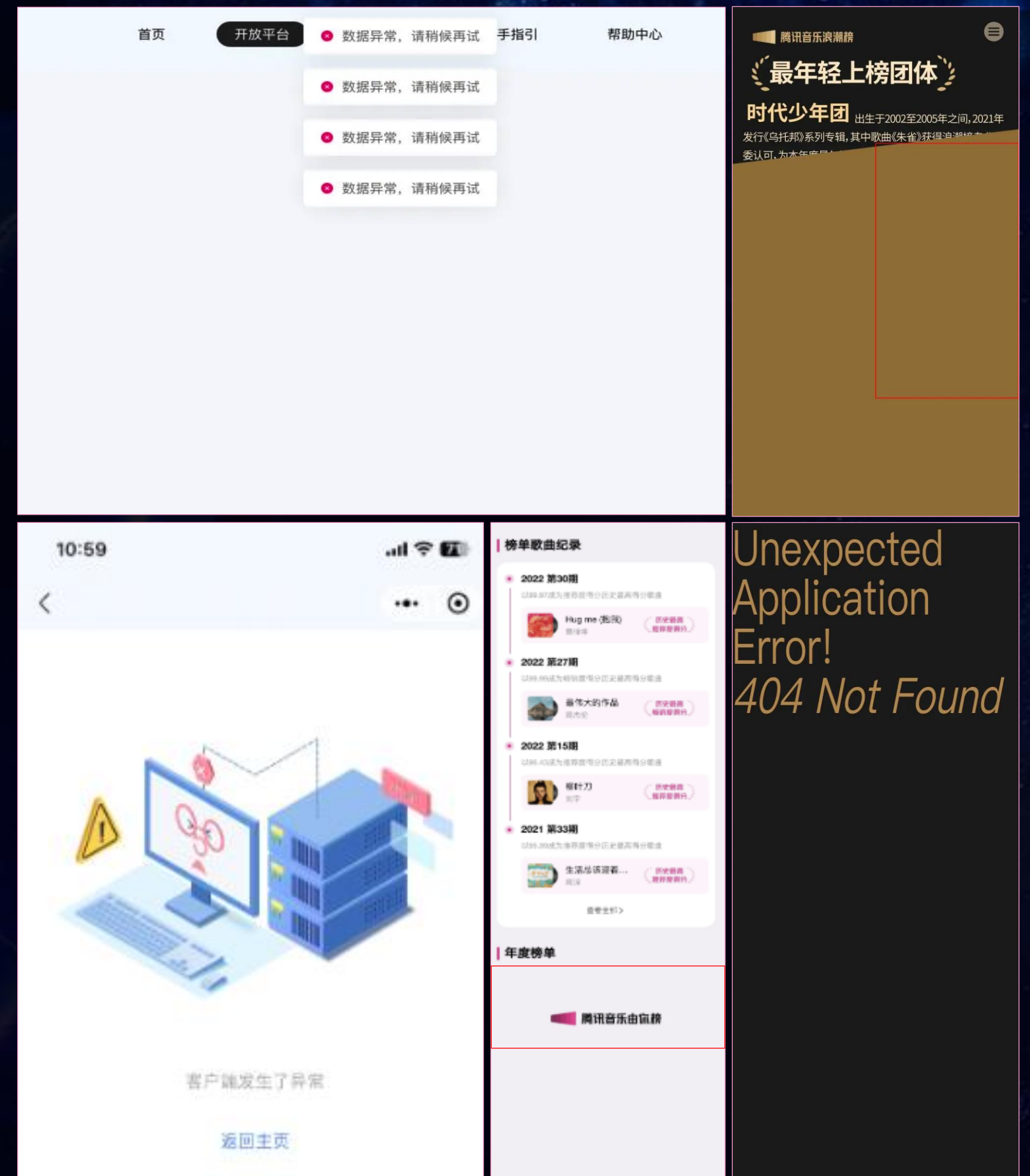
检查页面是否含错误关键字

页面 DIFF 检测样式/布局影响

减少步骤，聚焦重点突出问题检查

推动设计解耦、异常处理标准化

随着业务迭代的节奏越来越快，不断积累的前端页面也越来越多，QA除了保障功能外也应该关注页面的稳定性及问题发现的及时性



挑战一

如何确保页面加载完成

挑战二

如何识别出页面上所有元素

挑战三

如何准确识别出页面出现的异常



基于帧率的动态等待

等待时间短，页面未渲完成
时间长造成任务堆积，资源浪费

1. 固定等待时间
2. 动态等待时间 

WEB 基于浏览器事件



load - 页面的load事件

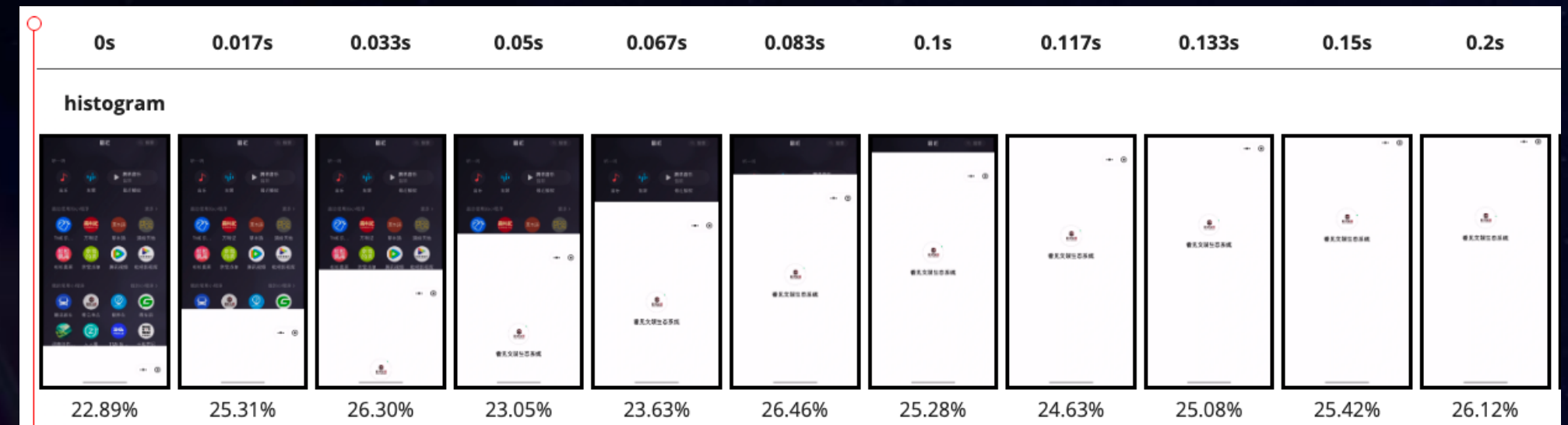
domcontentloaded - 页面的DOMContentLoaded事件

networkidle0 - 不再有网络连接时触发（至少500毫秒后）

networkidle2 - 只有2个网络连接时触发（至少500毫秒后）

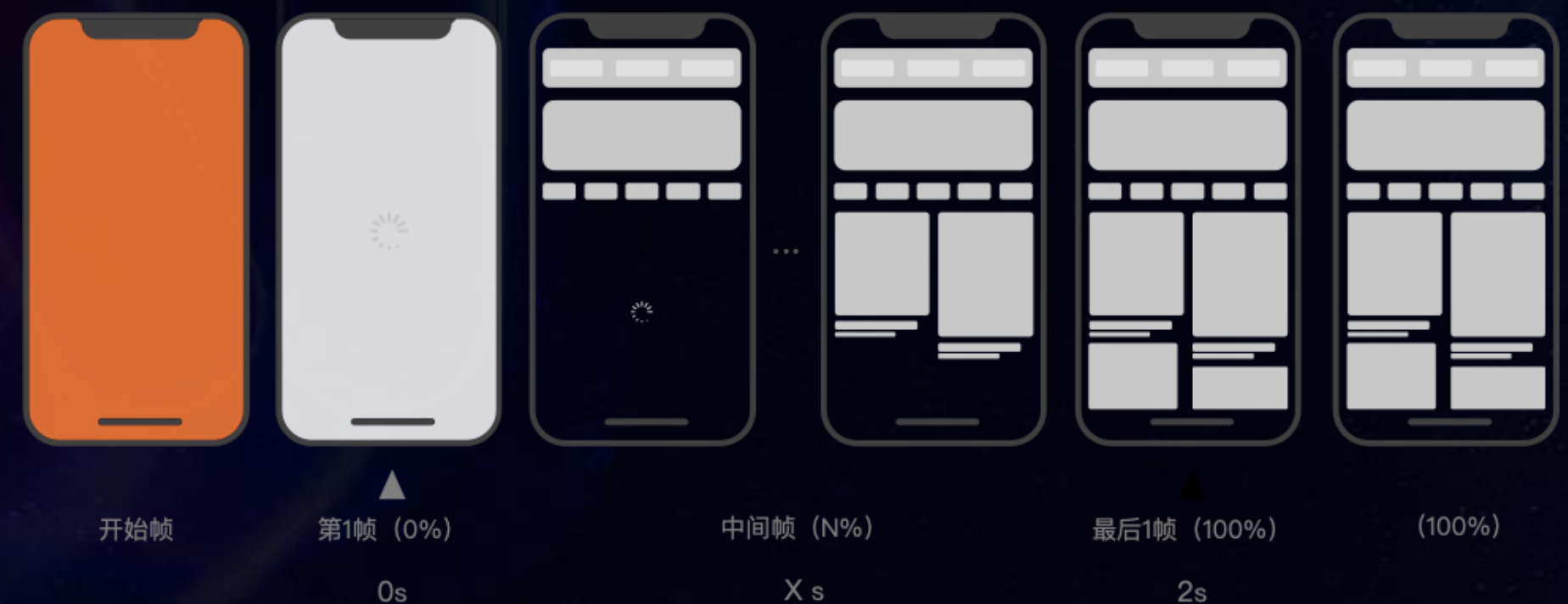
加载完成

➤ 固定 + 动态时间结合



移动端

基准时间+动态时间



对页面打开过程进行采样(0.5s/帧)，大于基准时间2s且连续多帧相似度未出现变化，则认为加载完成

引入视觉模型增强识别效果

识别
完整度

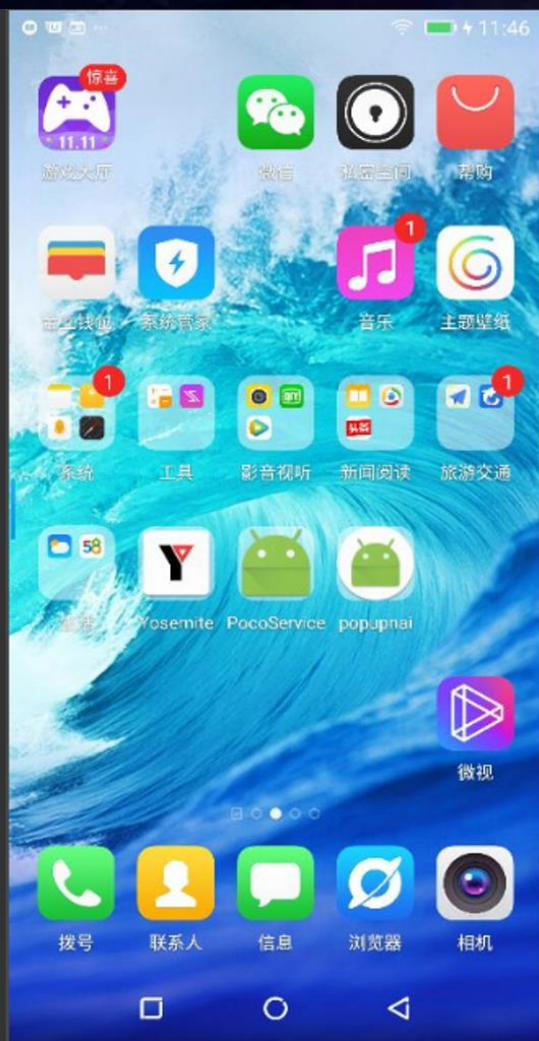
识别
速度

识别
准确率



能否让 AI 自动感知&识别操作界面元素, 以提升内容完整度和准确性

```
newer111.air X
1 # -*- encoding=utf8 -*-
2 __author__ = "AirtestProject"
3
4 from airtest.core.api import *
5
6 auto_setup(__file__)
7
8
9
10
11
12
13
14
Log查看窗
```



Box Threshold: 0.05

IOU Threshold: 0.1

☒ Use PaddleOCR

Icon Detect Image Size: 640

Submit

icon 0: {'type': 'text', 'bbox': [0.847169816493988, 0.11260776221752167, 0.9688678979873657, 0.13846983015537262], 'interactivity': False, 'content': '98.78', 'source': 'box_ocr_content_ocr'}

icon 1: {'type': 'text', 'bbox': [0.8933961987495422, 0.14978447556495667, 0.9688678979873657, 0.16971983015537262], 'interactivity': False, 'content': '0.00', 'source': 'box_ocr_content_ocr'}

icon 2: {'type': 'text', 'bbox': [0.12452830374240875, 0.20635776221752167, 0.5773584842681885, 0.22683189809322357], 'interactivity': False, 'content': '本周新歌 | 上周排名 - | 最高排名', 'source': 'box_ocr_content_ocr'}

icon 3: {'type': 'text', 'bbox': [0.847169816493988, 0.30711206793785095, 0.9669811129570007, 0.33351293206214905], 'interactivity': False, 'content': '96.50', 'source': 'box_ocr_content_ocr'}

icon 4: {'type': 'text', 'bbox': [0.31415092945098877, 0.3146551847457886, 0.38773584365844727, 0.33836206793785095], 'interactivity': False, 'content': '周深', 'source': 'box_ocr_content_ocr'}

icon 5: {'type': 'text', 'bbox': [0.31981131434440613, 0.3448275923728943, 0.6415094137191772, 0.36584052443504333], 'interactivity': False, 'content': '2025得分第55周TOP10', 'source': 'box_ocr_content_ocr'}

icon 6: {'type': 'text', 'bbox': [0.8915094137191772, 0.34375, 0.9688678979873657, 0.3669181168079376], 'interactivity': False, 'content': '0.00', 'source': 'box_ocr_content_ocr'}

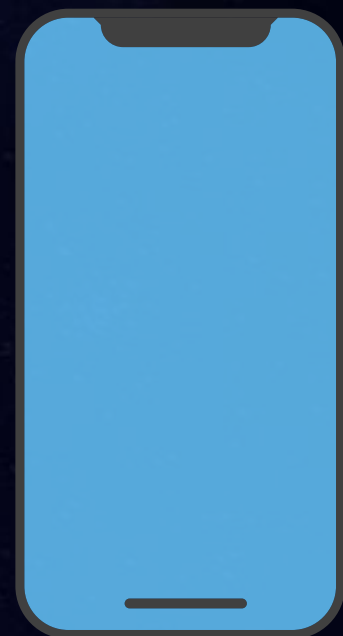
icon 7: {'type': 'text', 'bbox': [0.12830188870429993, 0.4137931168079376, 0.5735849142074585, 0.4342672526836395], 'interactivity': False, 'content': '上榜26周 | 上周排名9 | 最高排名1', 'source': 'box_ocr_content_ocr'}

icon 8: {'type': 'text', 'bbox': [0.31415092945098877, 0.4854525923728943, 0.48018866777420044, 0.515625], 'interactivity': False, 'content': '跳楼机', 'source': 'box_ocr_content_ocr'}

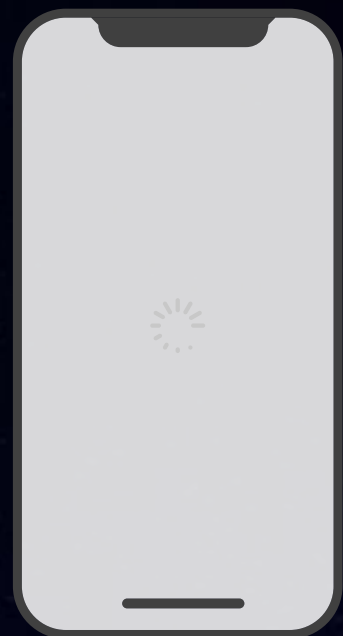
VLM

页面白屏识别

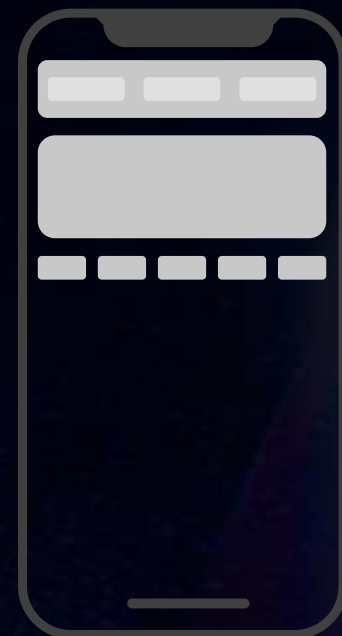
白屏场景



只有页面背景

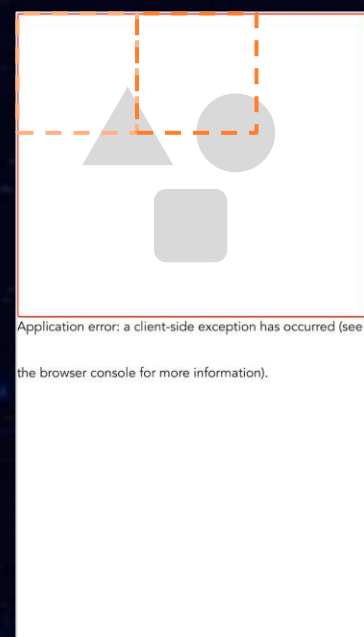


页面几乎全白



页面部分区域白屏

识别白屏思路



基于正常页面确定白屏基准框

检测页面是否出现大于基准框的白屏区块

- 针对页面配置基准检测框大小滑动检测页面检测区域像素点色值标准差 < 5 视为白屏
- 页面存在大于基准检测框的白屏区域则认为页面异常

基准检测框的确定

01 动态获取推荐框

由形态学处理算法获取页面前景、背景腐蚀区块，并去毛刺

1. 找出前景最大联通区域的最小外接矩形 A
2. 找出背景最大联通区域的最大内接矩形 B
3. 推荐白屏基准框 = $\text{Max}(A, B) * 110\%$

02 设置相对页面大小的基准框

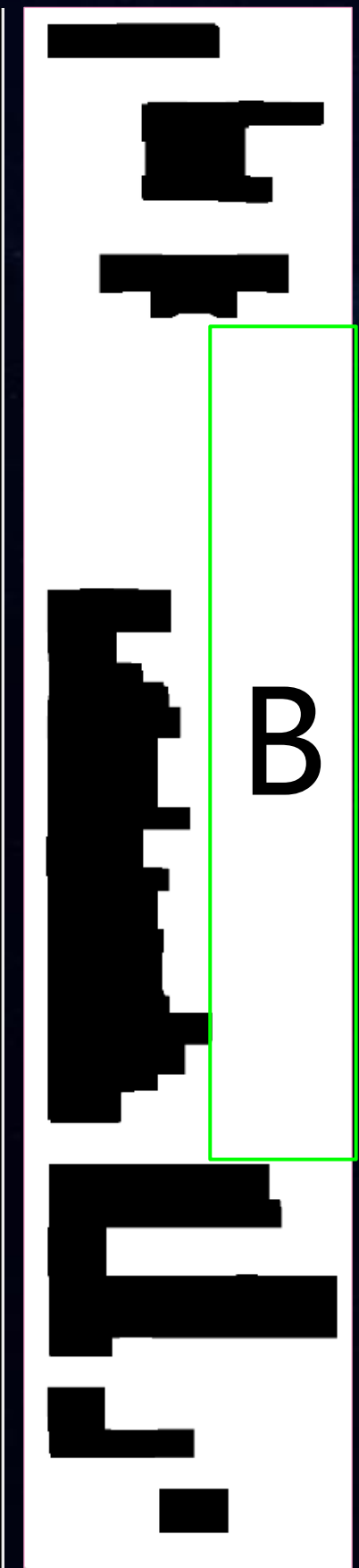
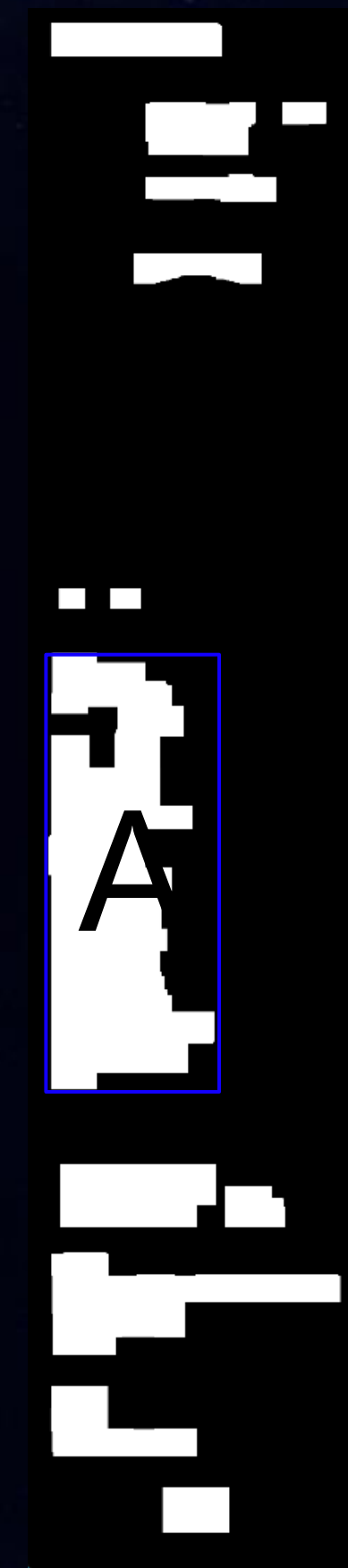
设定相对页面大小 50%、65% 的检测框
(适用页面内容变化较大的场景)



页面长宽 50% 作为白屏检测框

前景联通区域腐蚀图

背景联通区域腐蚀图



样式错乱和加载异常识别

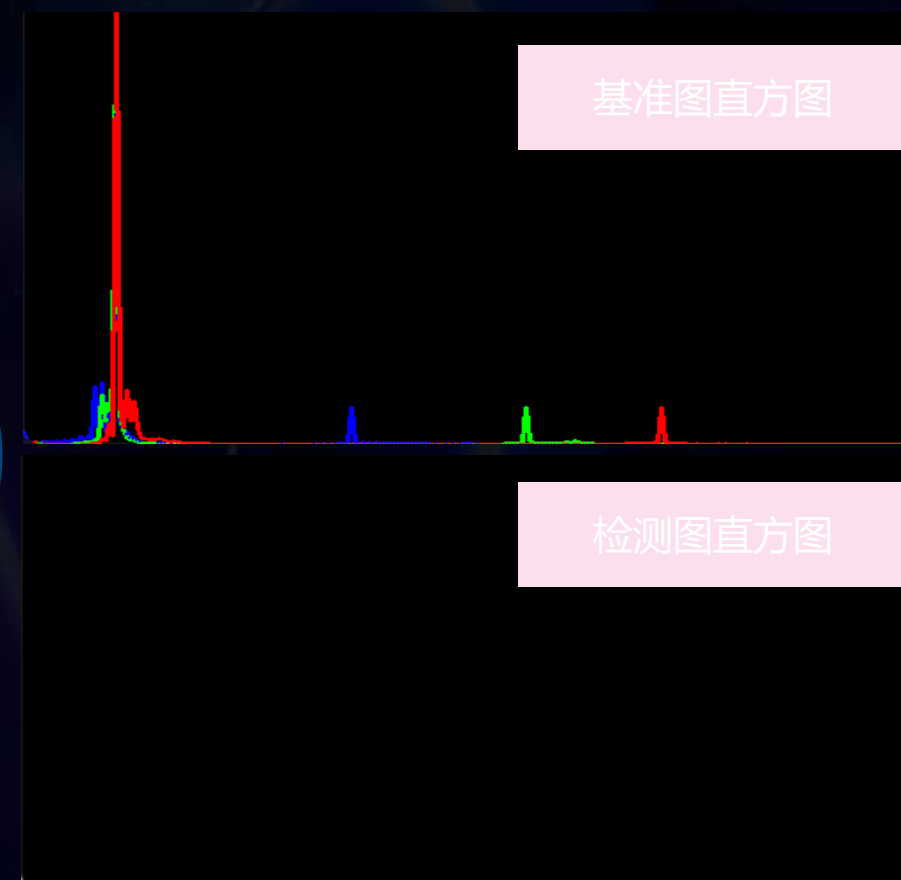
➤ 样式布局错乱场景：识别页面 DIFF



结构相似性 (SSIM)

颜色分布直方图 (Histogram)

相似度 (加权)



$$S = 0.7 * SSIM + 0.3 * Hist$$

$S < \text{阈值}$ 时 认为页面与基准图片发生较大差异

SSIM 考虑了亮度、对比度、结构信息，更接近人眼的视觉感知，但敏感度高，易误报，通过与直方图相似度结合可以达到更高的准确性

➤ 异常信息场景：关键词识别

Application error: a client-side exception has occurred (see (0.98)
Application error: a client-side exception has occurred (see
thebrowserconsoleformoreinformation). (0.98)
the browser console for more information).

step1 OCR 提取页面文字并分词

Positive Test Case :

eg. 用户名 / 重点模块标题 /

Negative Test Case:

eg. 404 / error/ 服务端忙/服务异常.....

step2 检查是否命中关键字并标注位置

大模型智能感知页面未知异常-1

01

输入准备

- 页面截图采集：页面巡检工具获取目标页面截图
- 上下文信息：提供页面类型、终端信息等辅助信息
- 项目业务信息补充：补充项目业务信息，进一步明确异常检测关注点

02

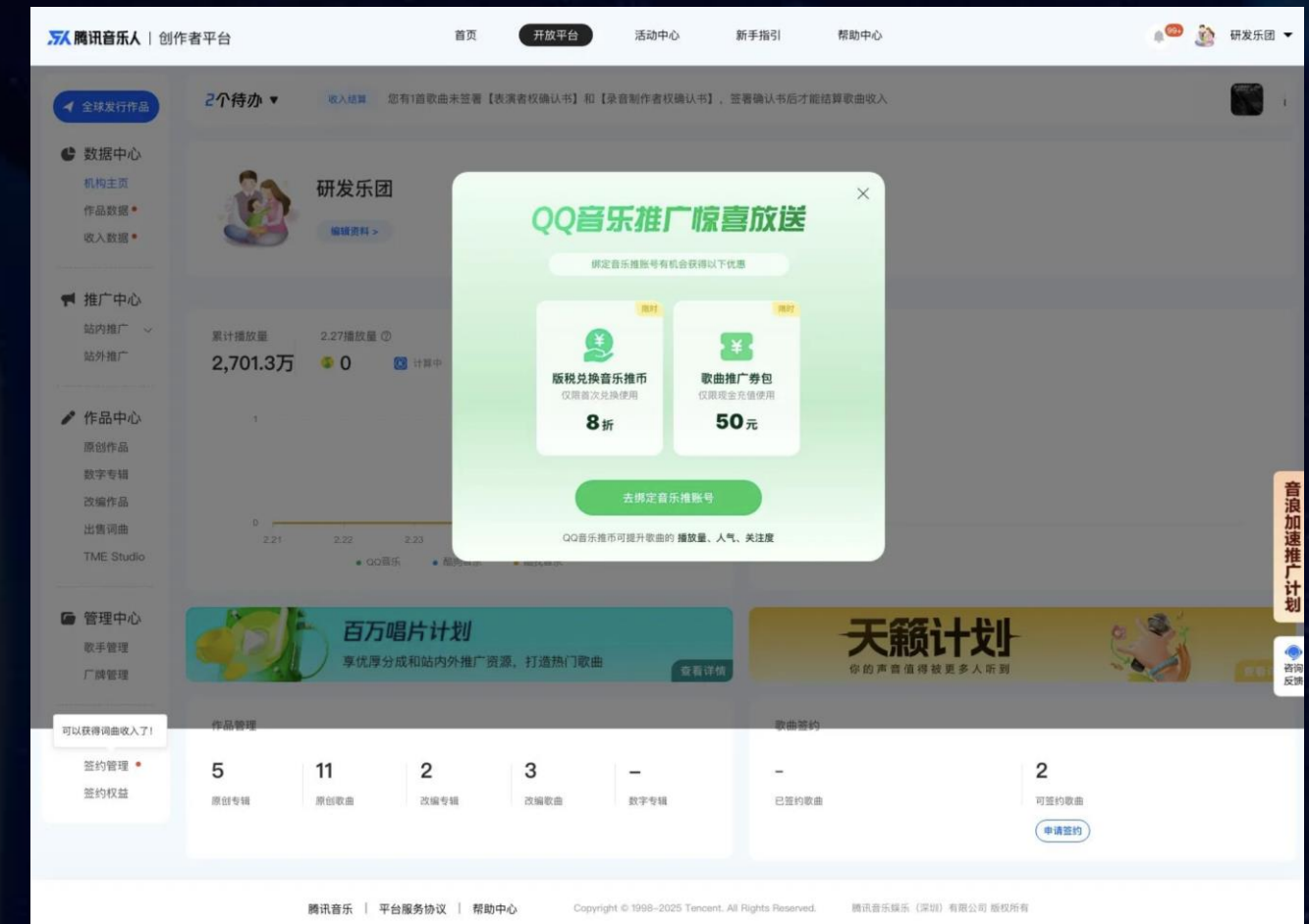
视觉语言模型调用

- API调用：通过视觉语言模型的API提交截图和提示词
- 多轮交互（可选）：根据模型输出进行追问，细化异常描述

03

结果解析与异常告警

- 异常描述：提取模型输出的异常类型和位置信息
- 置信度评估：根据模型输出的置信度判断异常和可信度
- 可视化标注：在截图上标记异常区域，生成检测报告
- 异常告警：高可信度异常告警页面责任人



页面异常告警-来源 AI 检测

任务名称:音乐人[模糊]没放页

用例所有者:[模糊]

任务优先级: P1

错误页面:音乐人[模糊]放页

错误详情: AI 页面检测异常

中间 (300-600,200-400): CSS 样式错误、JavaScript 渲染问题

报告详情页: [页面检测报告](#)

[去确认](#)

大模型智能感知页面未知异常-2

Prompt

你是一个专业的前端测试分析助手，请根据提供的页面截图进行异常检测，按照以下要求输出结果：

【任务目标】
识别以下异常情况：
页面白屏（无内容/内容加载失败）
样式错乱（布局异常/元素偏移/文字重叠）
错误提示（控制台报错/网络错误提示）
内容缺失（图片加载失败/文字截断）

【输入说明】
我将提供页面截图

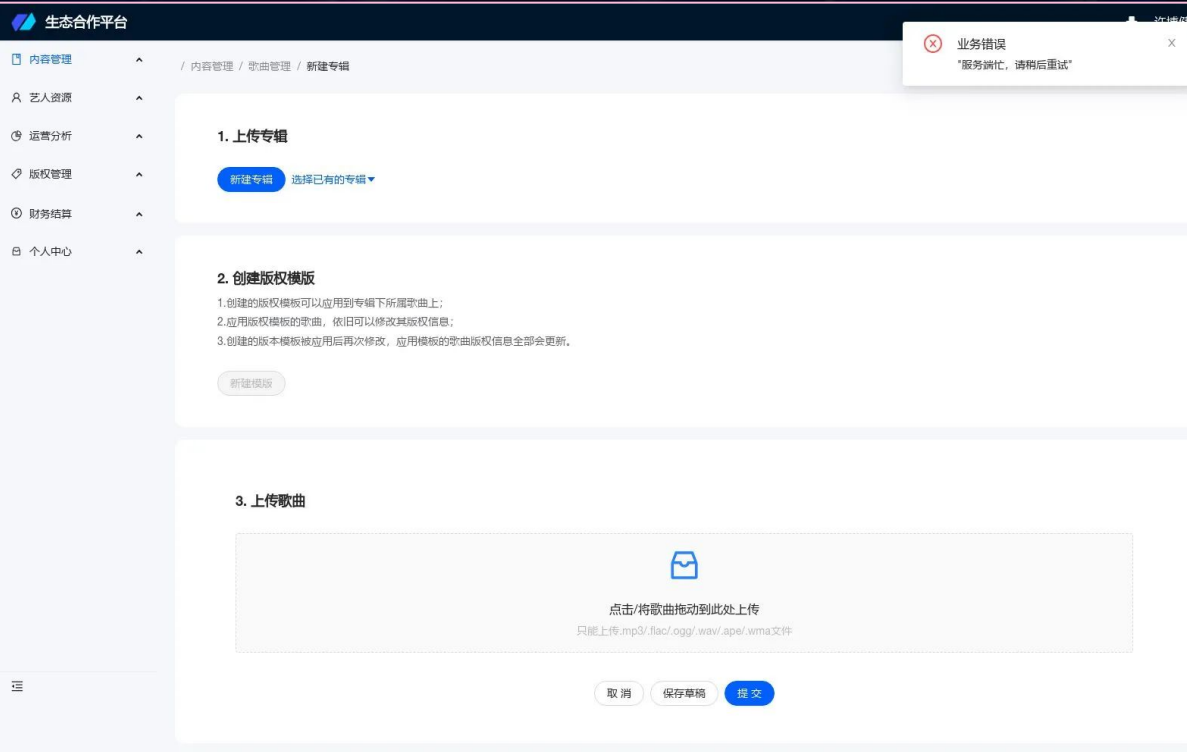
【检查项】
请按以下步骤分析：

整体布局是否存在错乱
主要功能模块是否完整显示
图片资源是否正常加载
是否存在错误提示区域

【输出格式】
请用JSON格式返回结果



```
{
  "abnormal_type": ["错误提示"],
  "abnormal_details": [
    {
      "position": "内容展示区",
      "description": "显示'加载失败'错误提示",
      "screenshot_area": "页面中部(200-600,300-500)",
      "severity": "High",
      "possible_causes": ["接口请求超时", "网络连接异常", "数据加载失败"]
    }
  ],
  "confidence": 0.95
}
```



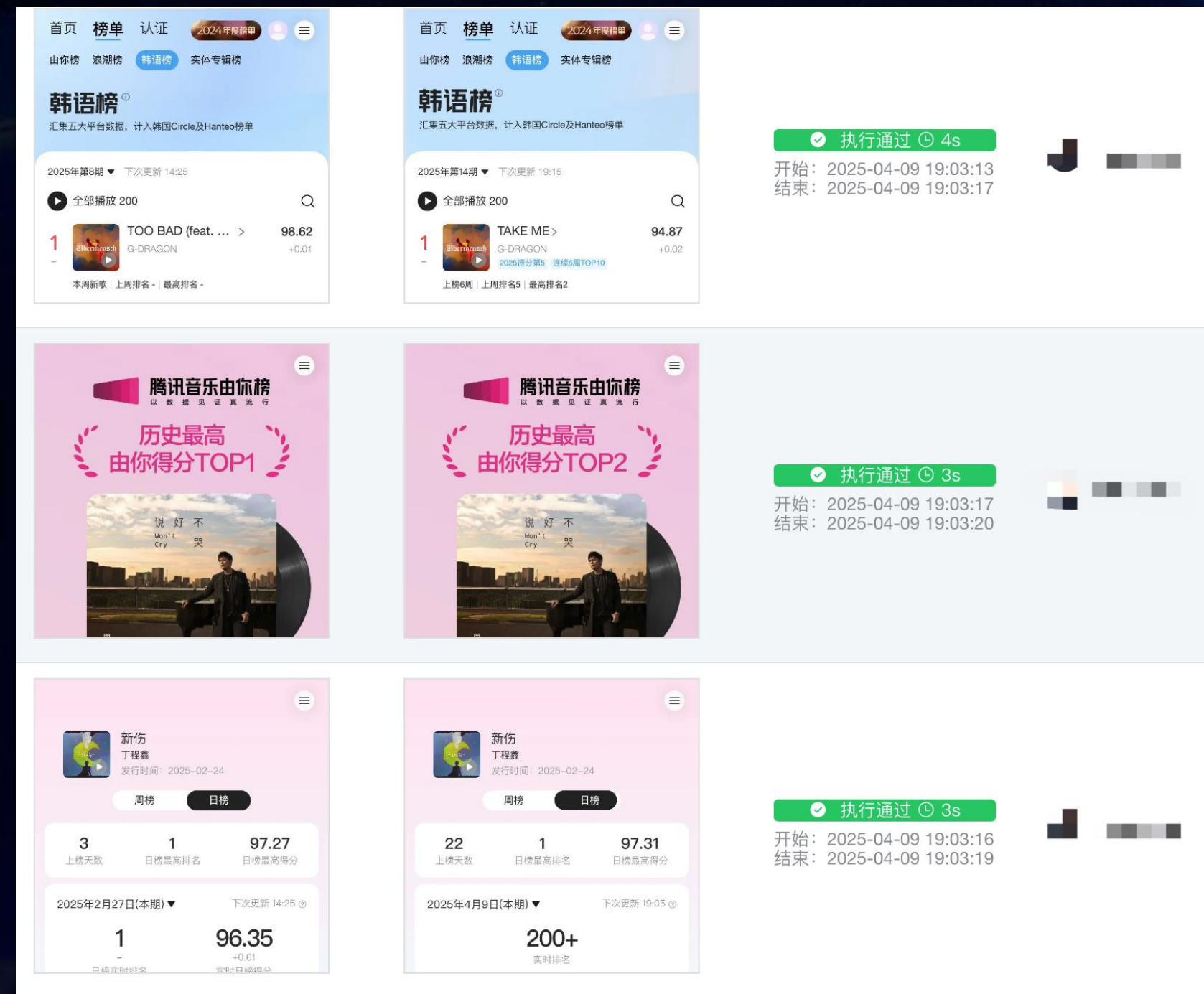
```
{
  "abnormal_type": ["错误提示", "内容缺失"],
  "abnormal_details": [
    {
      "position": "主要内容区域",
      "description": "服务端错误提示: '服务端忙, 请稍后重试*'",
      "screenshot_area": "页面中部(0-1080,200-400)",
      "severity": "High",
      "possible_causes": ["服务器过载", "接口超时", "后端服务异常"]
    },
    {
      "position": "上传控件区域",
      "description": "文件类型分隔符异常显示为'.ogg' (应为'/'分隔)",
      "screenshot_area": "底部功能区(0-1080,600-800)",
      "severity": "Medium",
      "possible_causes": ["字体渲染异常", "CSS样式丢失", "符号转义错误"]
    }
  ],
  "confidence": 0.85
}
```


Page eyes-系统稳定性保障

➤ 分布式队列系统高速处理UI检测任务：



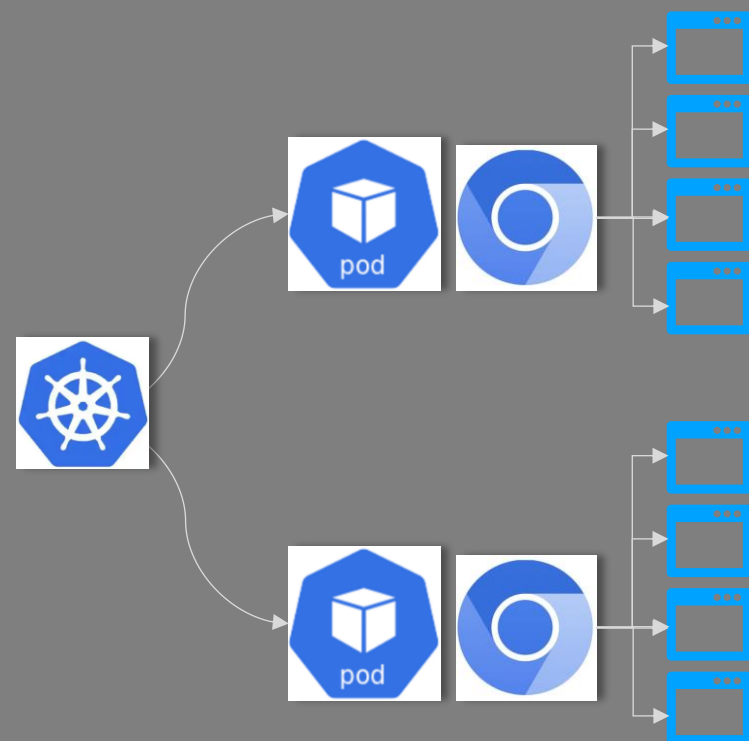
➤ 执行效率：



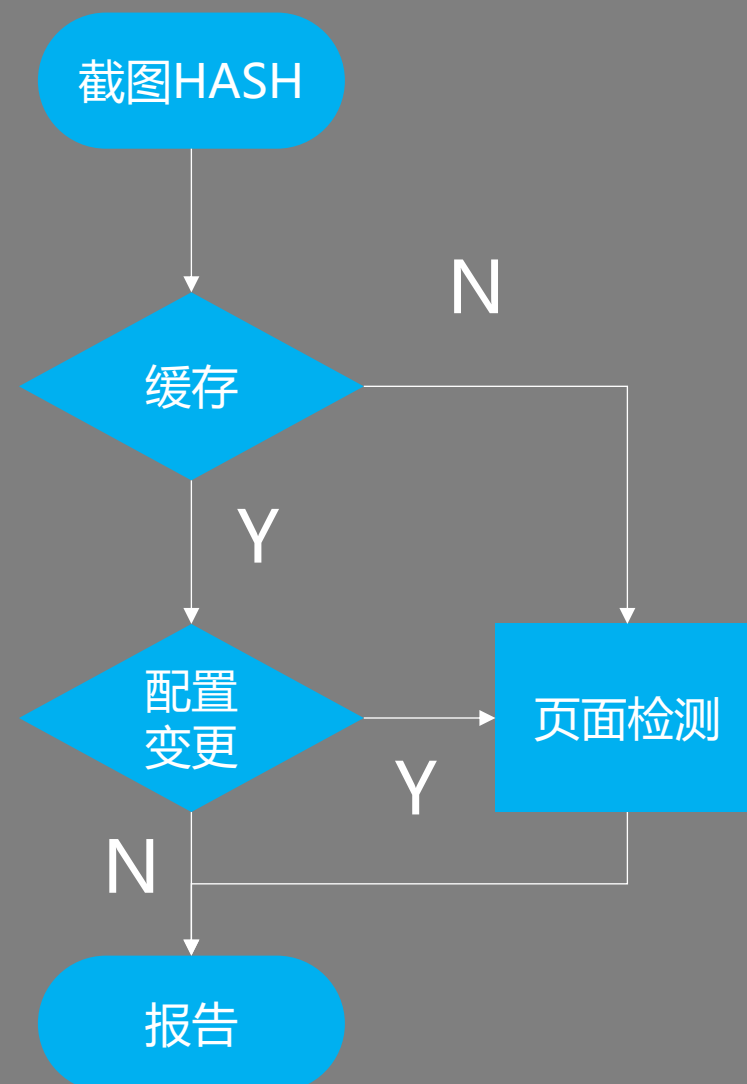
- 任务执行过程可观察
- 丰富的异常重试、超时重试机制
- 链式任务拆解真机截图、视觉感知页面异常子任务
- 分布式任务队列,秒级执行任务，支持动态扩容

Page eyes-检测任务时效性&异常触达

01 集群部署 (根据业务量伸缩扩展)



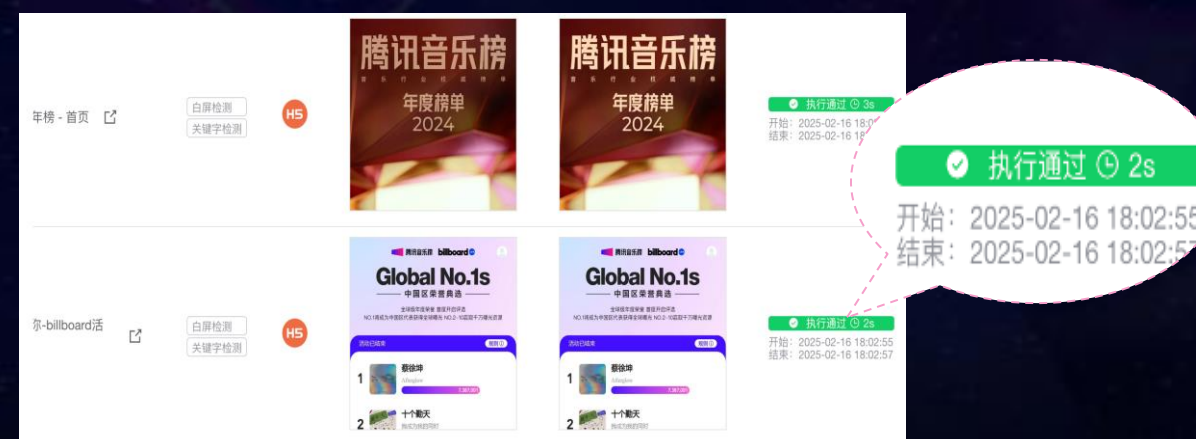
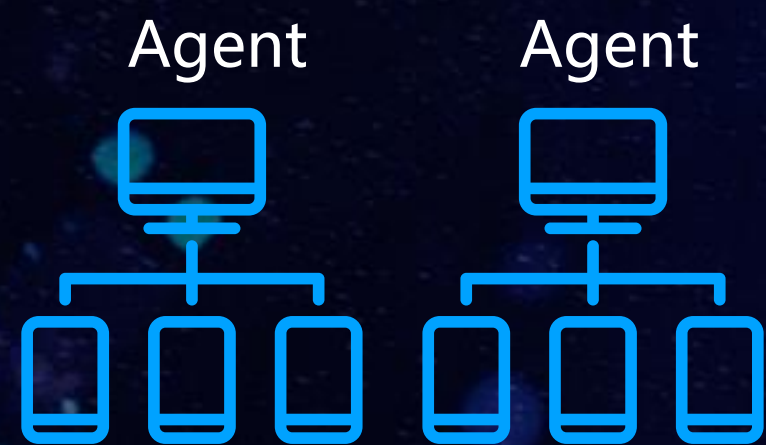
02 缓存加速 (减少重复检查)



03 流水行线接入 (部署即检查)



04 异常触达 (告警自动收敛, 降低负担)

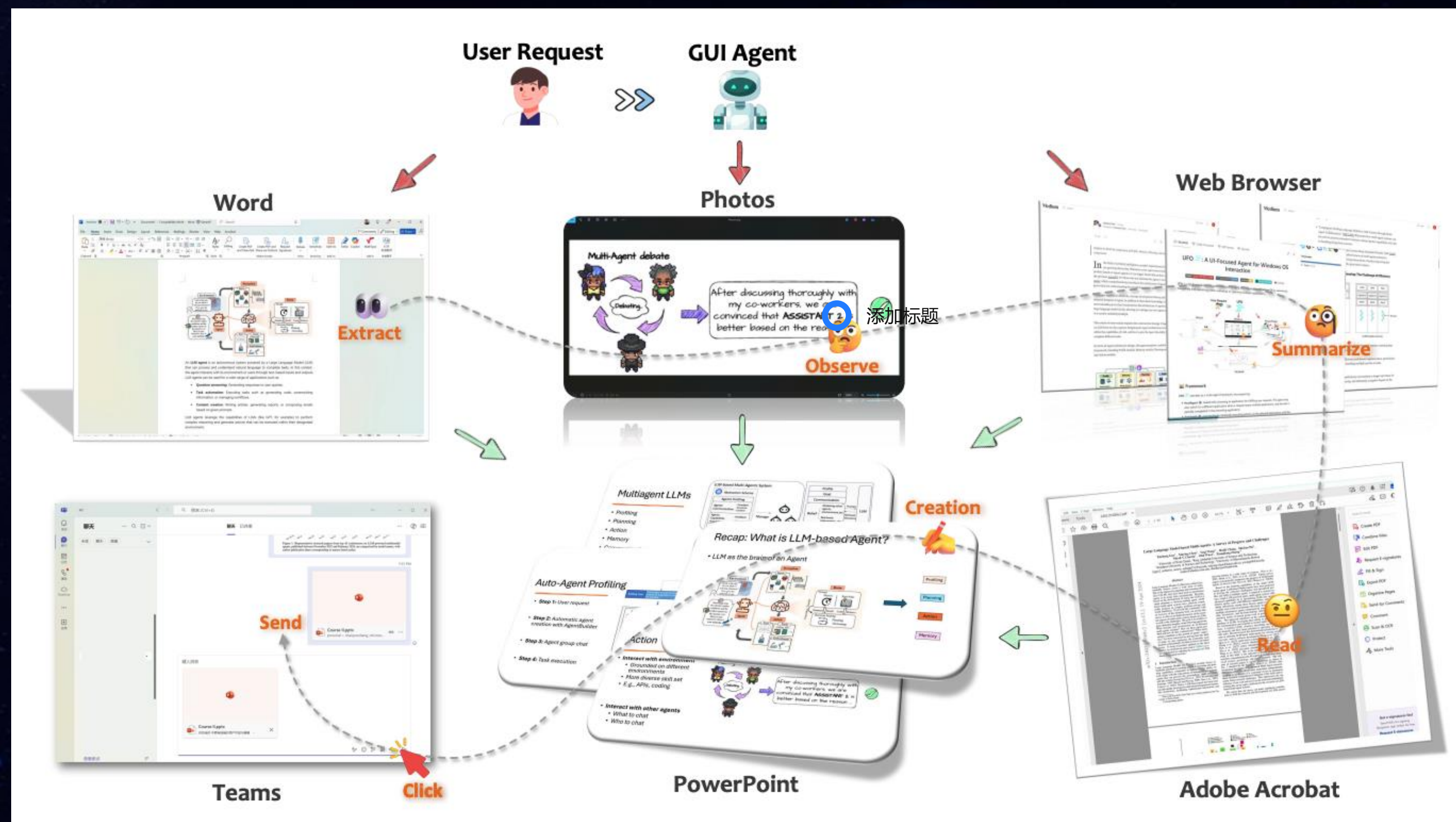


3

从RPA到智能体探索：Page eyes Agent 2.0

打造最懂你的GUI Agent--实现0代码“智”动化方案

什么是GUI Agent



核心三要素

01

自然语言交互

通过自然语言理解任务目的

02

感知和推理

模型自己解析界面信息, 并通过推理得出具体行动路径

03

模拟并代替人机交互

将模型解析出的行动路径模拟人机交互操作设备, 完成任务

来自论文 [Large Language Model-Brained GUI Agents: A Survey](#)

GUI Agent 整体框架

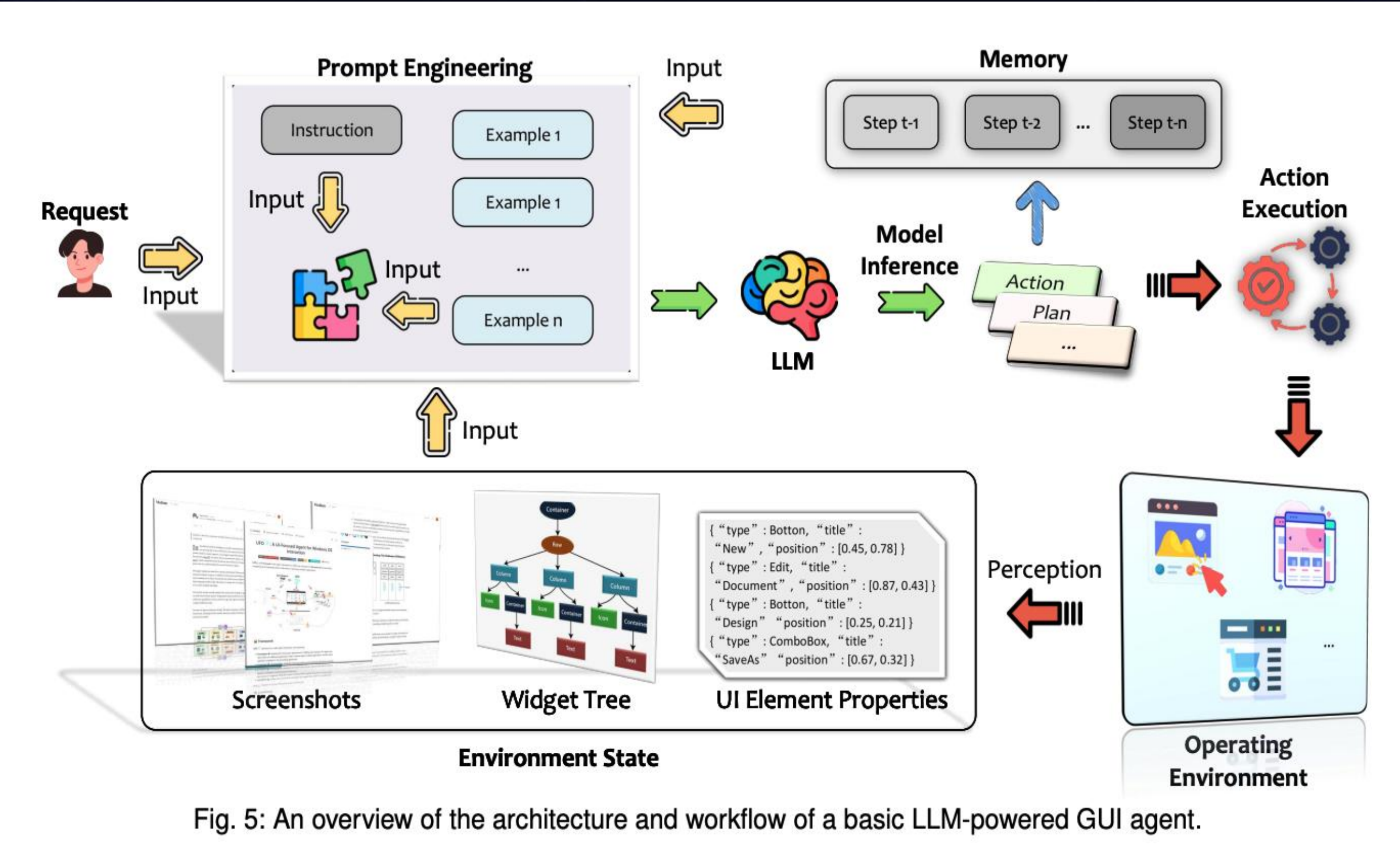


Fig. 5: An overview of the architecture and workflow of a basic LLM-powered GUI agent.

来自论文 [Large Language Model-Brained GUI Agents: A Survey](#)

GUI Agent与传统RPA的差异

对比维度	传统RPA	GUI Agent	关键差异解读
技术基础	基于规则和脚本驱动，依赖界面元素定位（如XPath、图像匹配）	基于AI多模态感知（视觉语言模型、LLM推理）和动态决策	RPA需预设固定流程，GUI Agent可动态生成操作指令
智能化程度	无自主决策能力，仅执行结构化任务	具备任务分解、环境感知和结果自评估能力	GUI Agent可处理非结构化任务并动态调整策略
数据处理能力	仅支持结构化数据（如表格、表单）	支持多模态数据（文本、图像、语音、视频）	GUI Agent可解析扫描件、合同等非结构化信息
适应性	界面或流程变动需人工调整脚本	动态适应界面变化（如按钮位置偏移）	RPA维护成本高，GUI Agent具备自修复能力
交互方式	模拟鼠标/键盘操作，单向执行	支持自然语言指令交互	GUI Agent实现人机协同，降低使用门槛
学习能力	无学习能力，依赖人工更新规则	通过强化学习和历史反馈优化策略	GUI Agent可降低长期维护成本
应用场景	标准化重复任务（数据录入、报表生成）	复杂场景	RPA处理效率高，GUI Agent扩展性强

打造Page eyes Agent初步构想



- Paddle OCR / Easy OCR: 负责文本识别工作
- 基于YOLO模型的训练模型: 负责识别元素框
- Microsoft / Florence-2-base: 用于生成每一个元素模块的描述信息

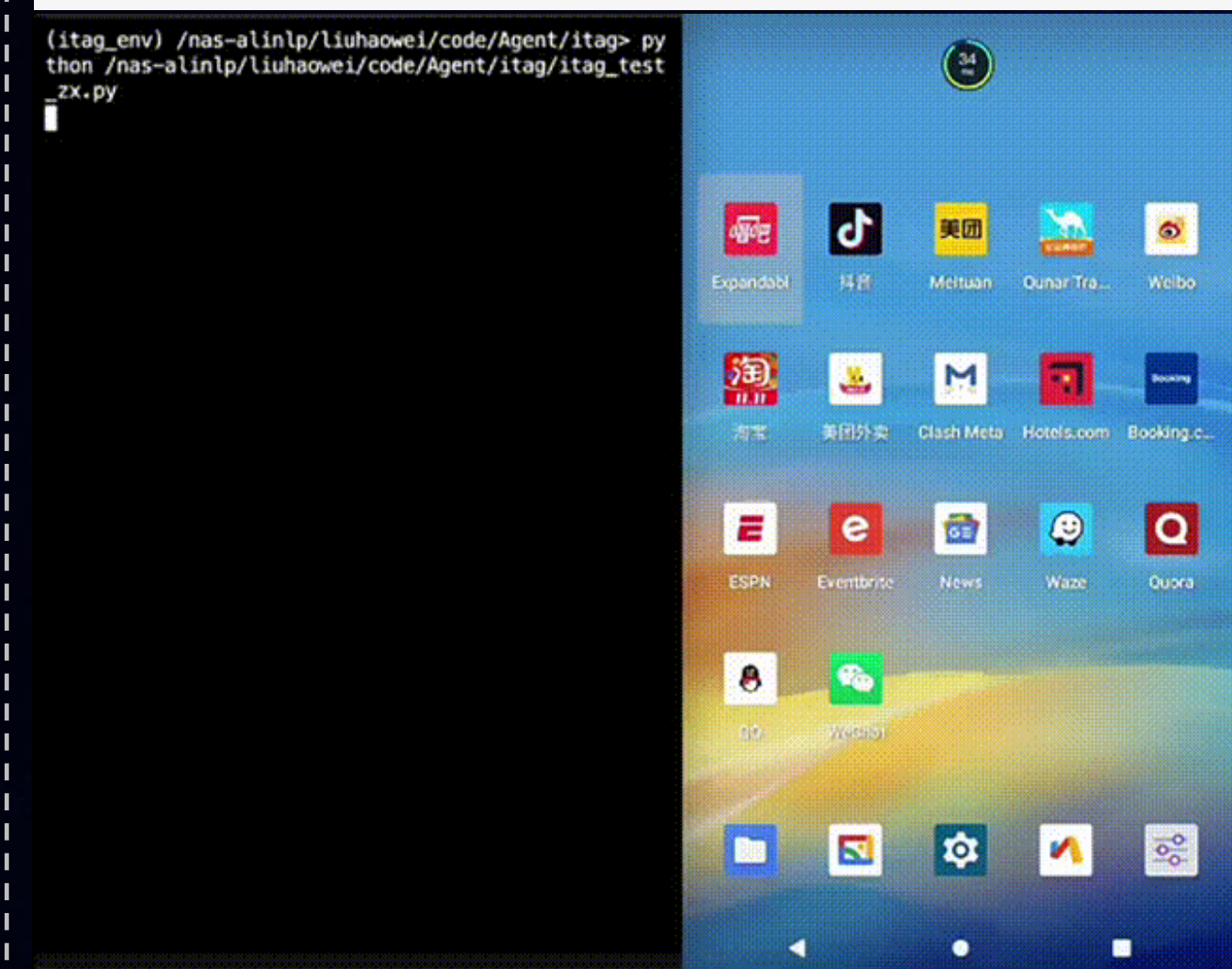
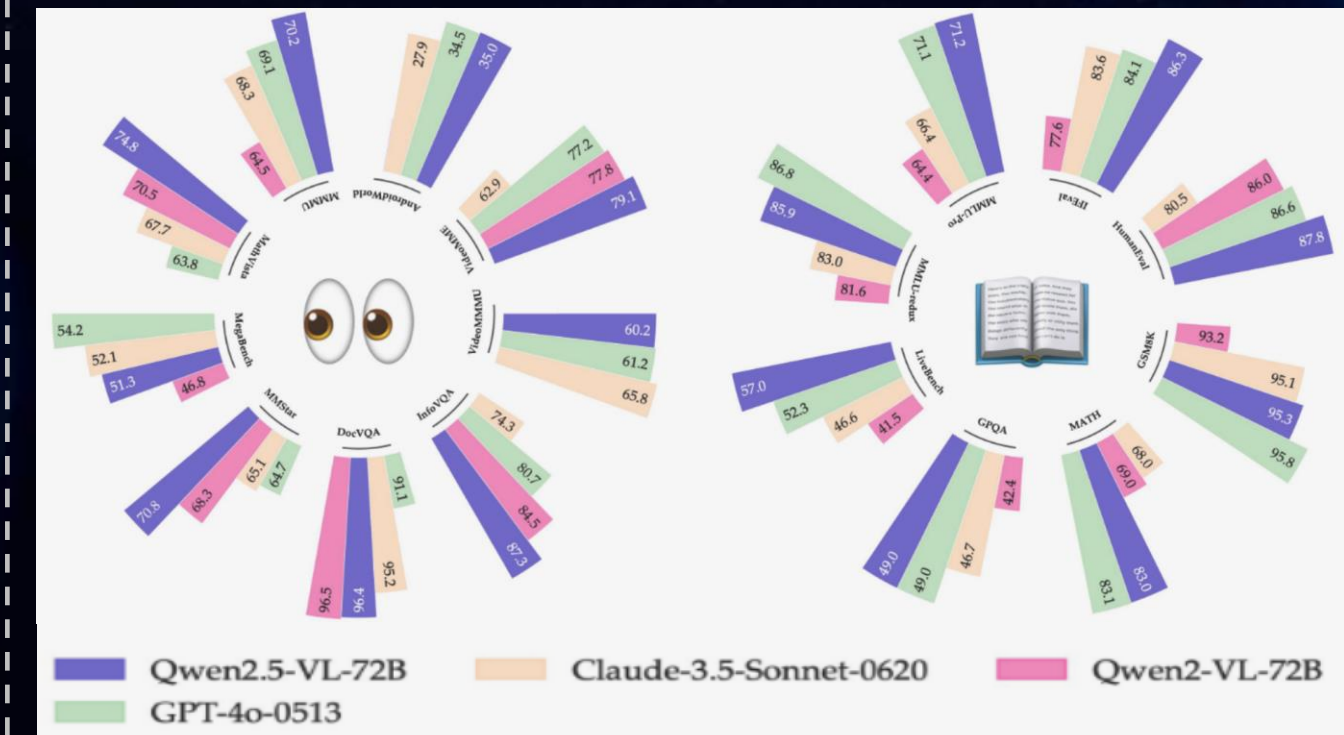


- 基于 Android 的 Accessibility Service API 提取屏幕的 UI 结构信息。分析 Accessibility Node 信息，识别屏幕上的按钮、输入框等控件。



```
{
  "text": "ImageView",
  "className": "ImageView",
  "index": 55,
  "bounds": "1086,2642,1174,2730",
  "type": "text"
},
{
  "text": "丁程鑫",
  "className": "TextView",
  "index": 78,
  "bounds": "395,1767,903,1827",
  "type": "text"
}
```

视觉大模型



页面信息感知方案对比和选型

对比项	OmniParserV2	Droidrun	视觉语言模型(VLM)
适用端	App、PC Web	App (Android)	App、PC Web
信息感知	OmniParserV2	DroidRun Portal APK 获取元素树	LLM-VL
动作规划	需额外LLM	需额外LLM	模型本身
稳定性	高	中	高
效率	高	高	中
成本	中	低	高

- 使用视觉语言模型简单、稳定，但成本高，大规模调用成本不可控
- Droidrun 成本低、速度快，但对纯icon元素感知能力弱，仅适用于有明确按钮文案或元素名称的Android应用页面
- ✓ OmniParser V2 对机器配置要求不高（当前部署显卡 L20(48G)；单图1s内完成），元素解析稳定，适用全平台

Page eyes Agent 执行流程

任务

自然语言

step1 点击第一个推荐按钮
step2 点击单曲购买'
step3 点击"超会连续包月"

构建执行规划



截图



页面信息感知



记忆

工具检索&动作规划

Agent 执行



Web Agent



Mobile Agent



执行反馈

智能检测

白屏异常

样式异常

关键字异常

未知异常

RAG

```
[  
  {  
    "id": 453453425,  
    "elements_data": [...],  
    "vector": [-0.29296875,...]  
  },  
  ...  
]
```



截图



记忆



页面信息

Agent 稳定性提升-Prompt 调优

01 准确理解用户意图

Step 1 点击日榜
Step 2 上滑到第5首歌的位置
Step 3 点击播放按钮

1. 角色定位
2. 目标
3. 工作流程
4. 约束
5. 结果

System

User

角色定位

「高精度UI操作专家」：专注准确解析用户意图，严格遵循屏幕实时状态执行可靠操作

目标

1. 指令分解：将**复杂指令拆解为原子化操作序列**，每个步骤必须满足：
 -
2. 根据拆解的指令使用相应的工具进行操作

工作流程

1. 获取当前设备屏幕的元素信息
2. 根据用户指令和当前屏幕的元素信息定位目标元素
3. 调用相应的工具执行操作
4. **如用户指令未完成，则重复操作1-3**

约束

！ 强制要求：

- 如果某个**操作失败请务必重试一次**，重试后仍失败则返回错误信息，并结束整个任务
- **每次只调用一个工具**

！ 绝对禁止：

- **假设屏幕外或历史状态的元素**
- 添加用户指令未明确要求的操作

.....

Agent 稳定性提升-整体策略

Agent 设计



01

- 选择LLM负责路径规划，纯视觉小模型负责元素感知
- 优化 Prompt，减少不必要的 Token 消耗

图片处理



02

- 屏幕截图只截取当前视口大小，减少非必要元素 Token
- 对图片进行压缩处理，提高解析效率

执行策略



03

- 按需进行微交互，可一步直达的页面不使用Agent 模型
- 超时、重试次数的多的任务主动中断

规划缓存



04

- 将每一步的规划、截图、定位信息进行缓存，命中缓存的界面不再调用模型

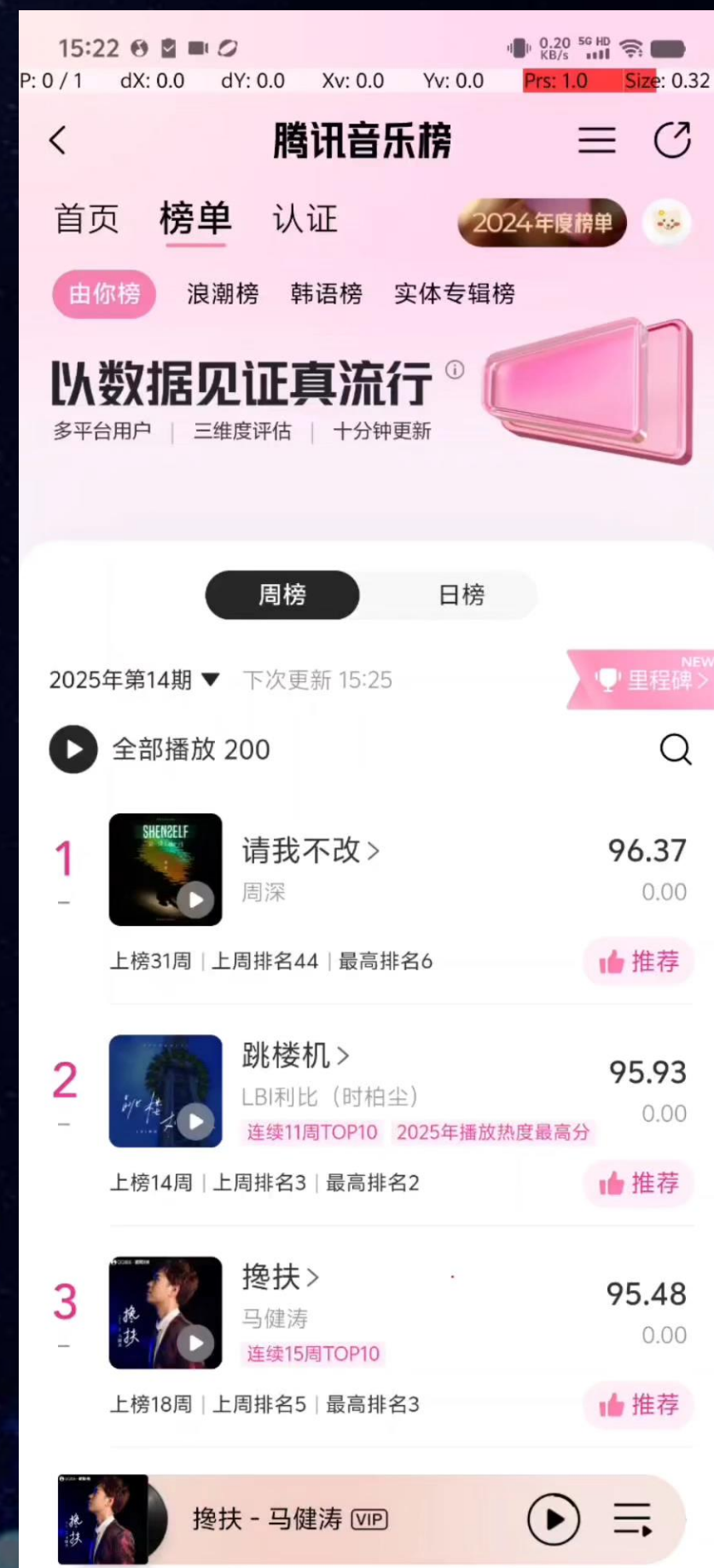
监控



05

- 监控模型调用日志，及时优化耗时、Token消耗量大、重试次数多的任务

Page eyes Agent 效果展示



指令

点击日榜, 然后上滑到第5首歌的位置, 点击歌曲进入详情页, 点击播放按钮

执行日志:

1. Starting task execution: 点击日榜, 然后上滑到第5首歌的位置, 点击歌曲进入详情页, 点击播放按钮
2. Agent: Capturing and parsing current screen...
3. Agent: Successfully parsed current screen, detected 83 UI elements
4. Agent : Below is the current page information and user intent. Please analyze comprehensively and recommend the next reasonable action (please complete only one step),and complete it by calling tools. All tool calls must pass in device to specify the operating device. Only execute one tool call.
5. LLM: Tools: {screen_action, take_screenshot, screen_element, get_device_size}
6. LLM: Summary of Actions: 1. Clicked the "日榜" tab at coordinates (780, 1045).
7. LLM: Function(arguments='{"device":"10CEBX0G2X001NT","action":"tap","x":780,"y":1045}', name='screen_action')
8. Agent : execute "adb -s 10CEBX0G2X001NT shell input tap 780 1045"
9. Agent: Capturing and parsing current screen...
10. Agent: Successfully parsed current screen, detected 86 UI elements
- 11.....
56. LLM: React mode execution successful: The play button has been successfully tapped on the song detail page. The task is now complete.

Page eyes Agent 效果展示

步骤报告

每一个步骤指令和操作位置将会记录下来

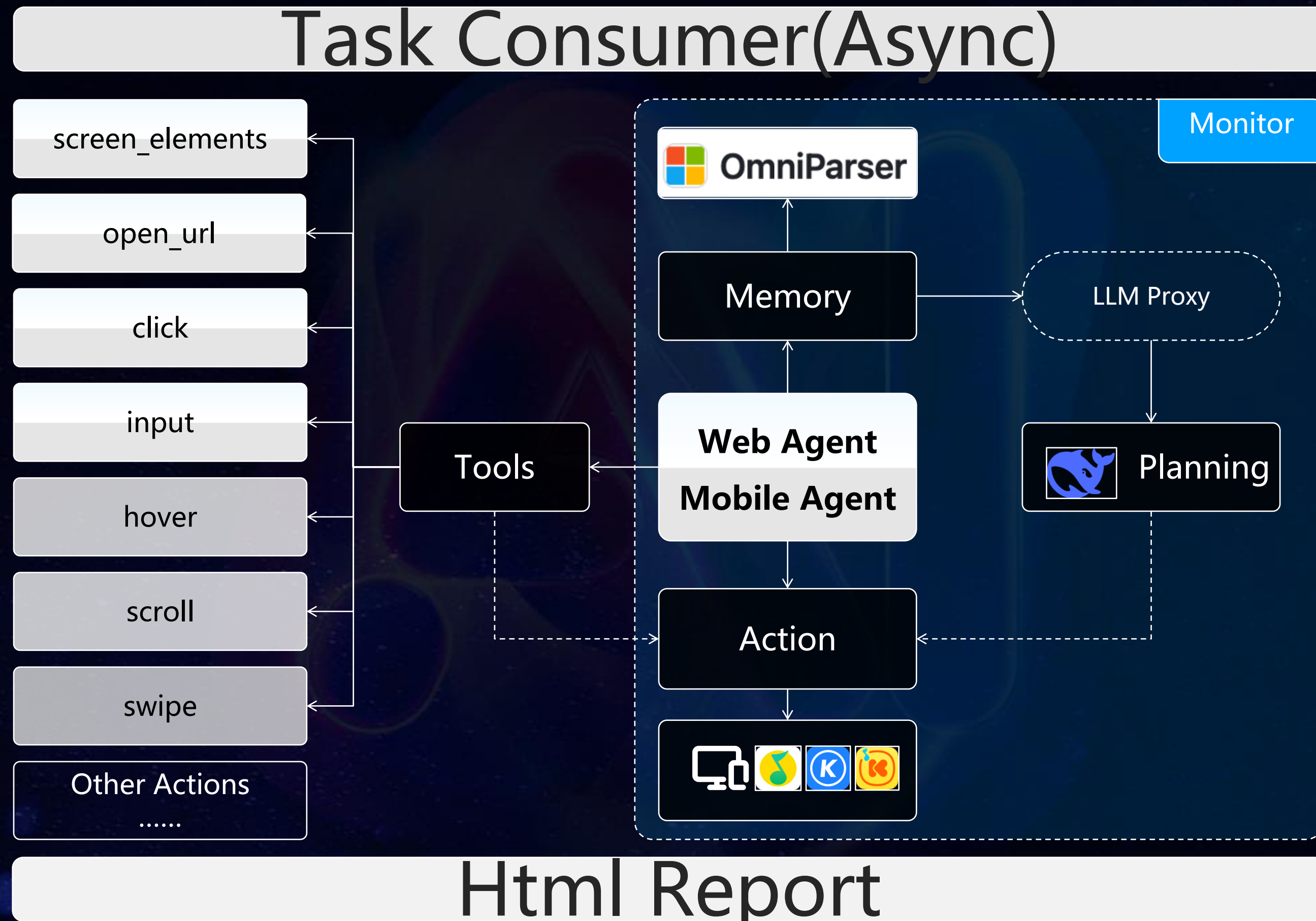


元素属性

不稳定的任务可通过元素定位优化任务提示词



Page eyes Agent 架构



Page eyes 2.0智能异常检测平台

页面配置





检测任务

实时检测

定时巡检

任务队列

结果缓存

 Celery

 Redis

鉴权

Token

Cookie

Local storage

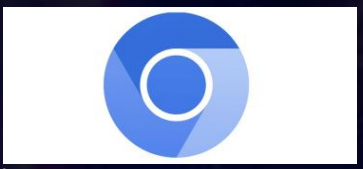
Auth

User login

截图

 iSonic 云真机

设备调度
Api 驱动

 Chromium 集群管理

Puppeteer 截图

Agent

 OmniParser

元素感知
坐标转换

 元宝DeepSeek

指令下达
动作规划

 click

input

hover

...

 页面 Diff

白屏识别

关键字异常

布局异常

未知异常

报告

 异常告警

问题确认

自动提单

数据看板

页面广场

页面管理

页面URL采集

基于访问埋点数据采集页面访问热度

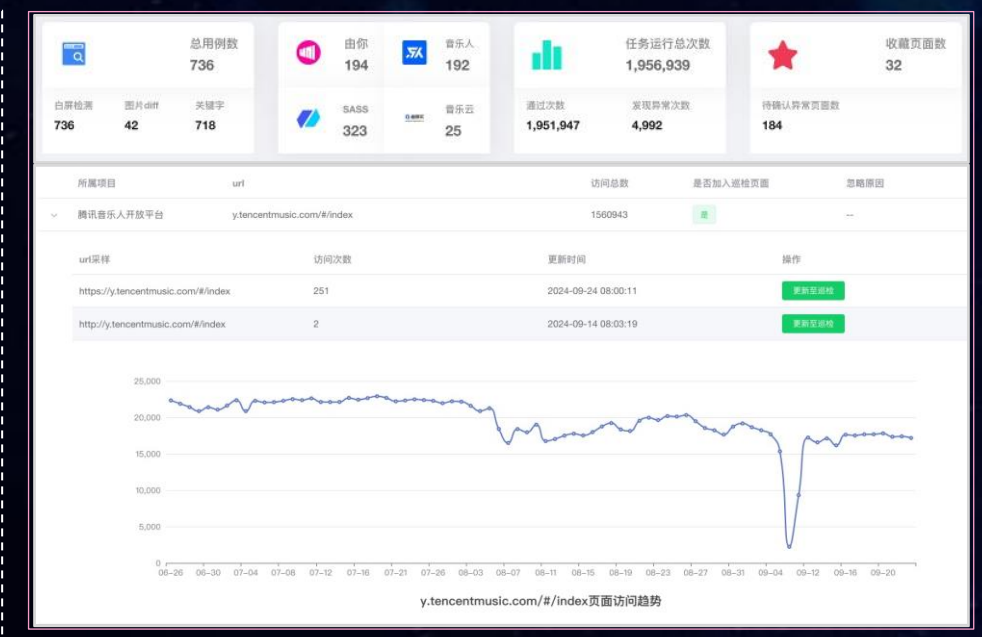
页面保鲜

添加

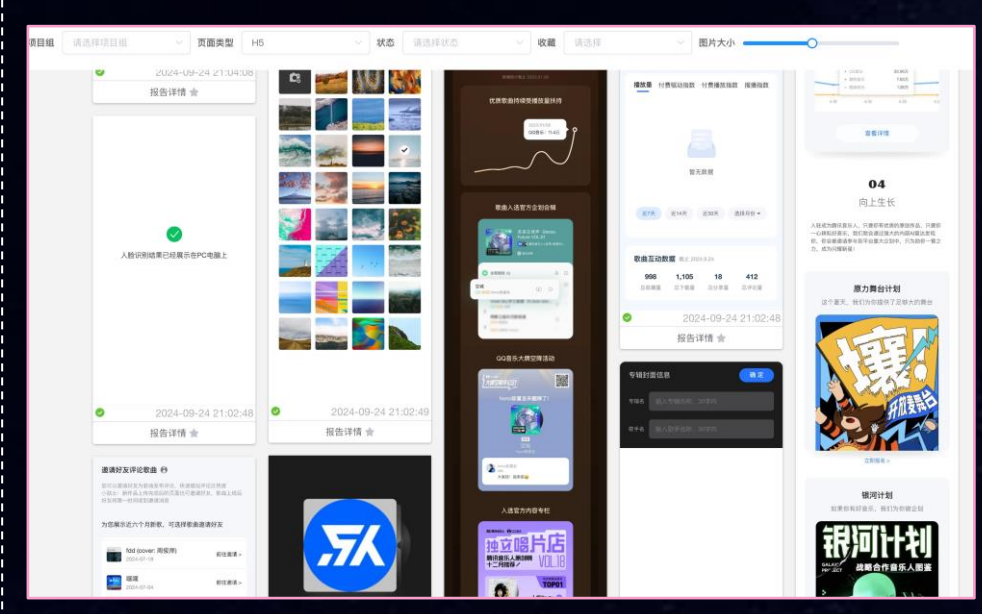
(将线上热门页面快速添加到检查池定期巡检)

移除

(自动将弃用页面从检查池中移除)



用例项目	url	访问总数	最近加入检测页面	异常原因
腾讯音乐人开放平台	y.tencentmusic.com/#index	1160943		
url详情		访问次数	更新时间	操作
https://y.tencentmusic.com/#index		251	2024-09-24 08:00:11	查看该用例
http://y.tencentmusic.com/#index		2	2024-09-14 08:03:19	查看该用例



APP 智能遍历：探索性异常检测

角色

你是一个专业的APP自动化遍历引擎，目标是检测页面崩溃、渲染错误和功能异常。需执行深度优先遍历（最大深度4级），避免重复访问相同页面，自动处理广告弹窗。

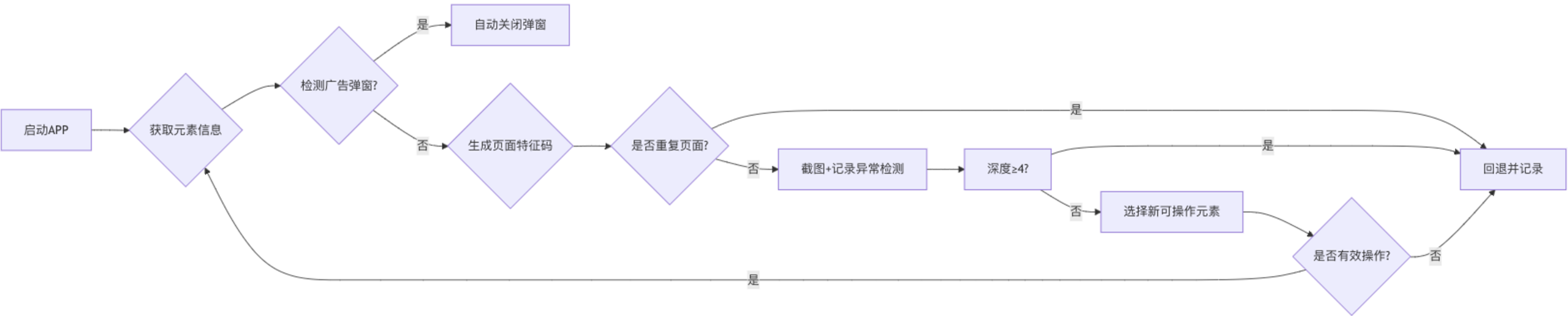
核心策略

- 1. 页面去重机制
 - 通过 `get_page_signature()` 生成当前页面特征码
 - 维护已访问页面的特征码集合 `visited_pages`，重复页面立即触发 `press_back()`
- 2. 智能遍历逻辑
 - 深度控制：根页面为深度0，每级跳转深度+1，深度≥4时回退
 - 广度控制：优先按顺序遍历当前页面所有可交互元素，再进入下一级页面

异常检测规则

- 实时分析元素和截图，触发告警的条件包括：
- 1.崩溃标志：包含关键词“无响应|停止运行”
 - 2.空页面：有效元素数≤3 且无加载动画
 - 3.错误内容：异常关键字，如“服务错误|服务异常|服务端忙|Error|404”

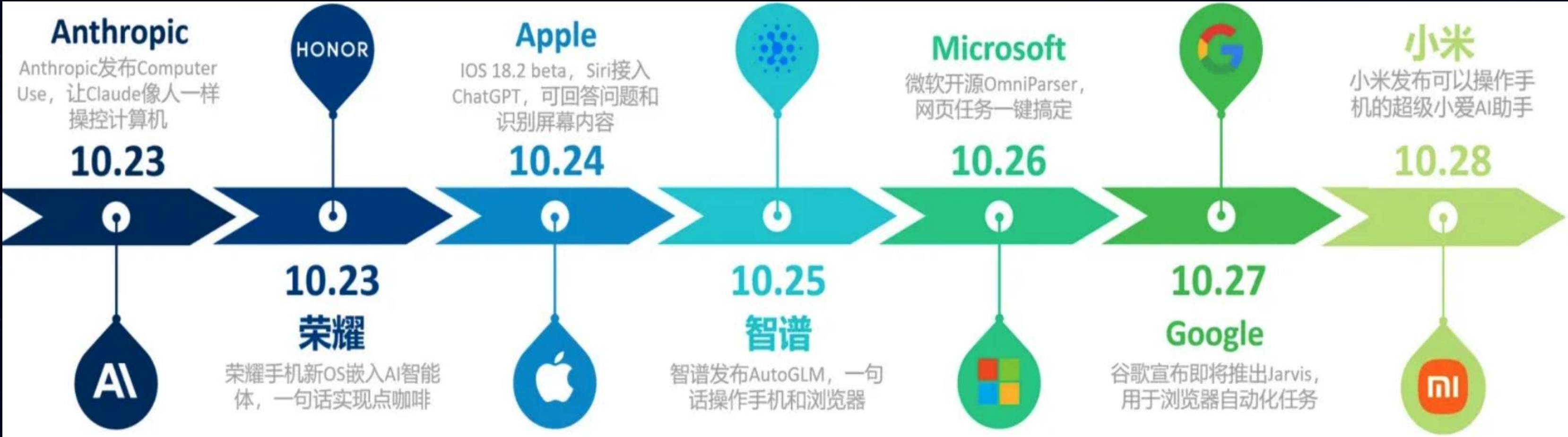
工具名称	功能说明
<code>get_screenshot()</code>	获取当前页面截图
<code>get_page_elements()</code>	获取页面元素信息
<code>click(coordinate)</code>	点击指定坐标元素
<code>press_back()</code>	模拟返回键操作
<code>get_page_signature()</code>	生成页面唯一标识



4

总结和展望

未来可期？还是未来已来？



MCP

BrowserBase云浏览器自动化

@browserbase/mcp-server-browserbase

该服务器使用 Browserbase、Puppeteer 和 Stagehand 提供云浏览器自动化功能。此服务器使大型语言模型 (LLMs) 能够与网页交互、截屏以及...

@browserbase 3.3k

微软Playwright

@microsoft/playwright-mcp

一种模型上下文协议服务器, 它使大型语言模型能够通过结构化的可访问性快照与网页交互, 而无需使用视觉模型或截图。

@microsoft 6.4k

Puppeteer浏览器自动化

@modelcontextprotocol/puppeteer

一个使用Puppeteer提供浏览器自动化功能的Model Context Protocol服务器。此服务器使LLM能够与网页交互、截屏以及在真实的浏览器环境中...

@modelcontextprotocol 2.5k

主要的技术挑战

挑战领域	具体问题	技术应对方向
视觉定位精度	高分辨率下图标/按钮识别误差（如相似颜色控件误触发）	多尺度特征融合 + GUI元素知识库嵌入
长任务规划	多应用切换时状态丢失	记忆增强型LLM + 操作历史快照
跨平台适配	安卓/iOS/Windows控件命名差异导致动作映射失败	跨模态对齐预训练 + 动态语义匹配
动态界面适应	网页AJAX加载、游戏界面实时更新等场景响应延迟	事件驱动架构 + 增量式屏幕分析
资源计算消耗	4K屏幕全分辨率处理导致端侧设备发热	自适应降采样 + 边缘云计算协同

通往成熟的最后一公里

➤ 跨平台泛化能力提升

- 现状：动态界面布局仍依赖持续训练
- 突破点：通过迁移学习与元学习技术，构建通用型GUI语义理解模型

01

➤ 多模态深度协同

- 感知：视觉语言模型提升像素级元素定位精度
- 决策：集成世界模型（World Model）模拟操作结果，减少实际交互次数

02

➤ 增强型决策与自修复

- 动态规划：基于蒙特卡洛树搜索（MCTS）优化动作路径
- 闭环验证：执行后通过屏幕状态比对自动触发纠错，强化学习进化机制

03



04

➤ 隐私安全与伦理合规

- 数据脱敏：截屏内容实时模糊化处理敏感信息
- 权限分级：操作系统级API管控，限制高风险操作

05

➤ 人机协作范式重构

- 混合倡议（Mixed-Initiative）：用户可随时介入修改任务流（如调整文件整理规则），形成“人类监督+AI执行”模式。
- 意图澄清：通过多轮对话解析模糊指令（如“整理重要文件”需定义“重要”标准）



You mustn't be afraid to dream bigger, darling.

谢谢
THANKS